University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Data Structures & Algorithm	**Course Code:** CMPE-212L
Assignment Type: Complex Engineering Problem	**Dated:** 19th March 2024
Semester: 4th	**Session:** 2022
Lab/Project/Assignment #: 1	**CLOs to be covered:** CLO 2
Lab Title: Expression Evaluator	**Teacher Name:** Engr. Afeef Obaid

Complex Engineering Problem (CEP):

No.	Attribute	Details
WP1	Depth of Knowledge required	To create a program that converts infix expressions to postfix and evaluates them, you'll need a depth of knowledge of data structures like stacks, an understanding of infix, postfix, and prefix notations, and knowledge of operator precedence and associativity. Additionally, proficiency in string manipulation, error handling, and mathematical operations is essential for developing a robust solution.
WP2	Range of conflicting requirements	In developing the program, conflicting requirements may arise between optimizing efficiency for performance and maintaining readability, as well as between thorough error handling and achieving high execution speed. Balancing these aspects is crucial to ensuring both functionality and usability of the solution.
WP3	Depth of Analysis required	Developing the program requires in-depth analysis of algorithms for expression conversion and evaluation, as well as thorough consideration of error handling, performance optimization, and user input validation to ensure reliability and efficiency.

CEP Description

We talked about solving mathematical expressions. Now the common mode of expression specification is in infix format, which is difficult to process due to the varied number of arguments to operations, precedence of operations and brackets in expressions. Therefore, infix expressions are converted to prefix or postfix expressions before evaluation. Therefore, the target of this problem is to make a program that takes an infix expression from the user and evaluate it to report the solution of that.

Tasks:

- Convert infix expression to corresponding postfix expression.
- Evaluate the postfix expression for numerical result.

Input: Mathematical Infix Expression taken as string

Output: Equivalent Postfix Expression, Numerical answers

Allowed operators: + - / * all brackets

For example:

Input: 2+(5/3)*9

Output: 2 5 3 / 9 * +

For simplicity, solution can be implemented for single digit values only. Bonus: handling multi digit input like 2 + 15 / 5 * 9 will score better.